ORIENTATION

Programming for Information Systems

Welcome: purpose, people and a first project

1

Meet each other

Introduce yourself and one learning goal.

2

Understand the course

Explore its purpose and the system we will build.

3

Find your route

Learn how sessions, materials and support fit together.

4

Prepare to start

Check expectations, assessment and your setup.

10 weeks · three 90-minute sessions each week · 45 contact hours.

Lecturer: Mengsay Loem

Part-time Lecturer

2026–present

National University of Battambang

Cambodia

Programming for Information Systems.

Engineer

2026–present

Preferred Networks, Inc.

Japan

Building large language models (LLMs).

Researcher

2024–2026

Sansan, Inc.

Japan

R&D on AI for document understanding and information extraction.

Education

Tokyo Tech

Japan

M.Eng. in Artificial Intelligence
B.Eng. in Computer Science

Outside work: hiking, walking, sauna and hot springs.

Your turn: a quick introduction

Your name

What name would you like us to use?

Your starting point

Have you tried programming? “Completely new” is a useful answer.

One learning goal

What would you like to build, understand or get better at?

One person at a time · about 30 seconds each · short answers are welcome.

The idea behind this course

1

Understand a need

Who needs help? What decision is difficult?

2

Make it explicit

Define information, units and rules.

3

Build small parts

Turn the rules into working Python.

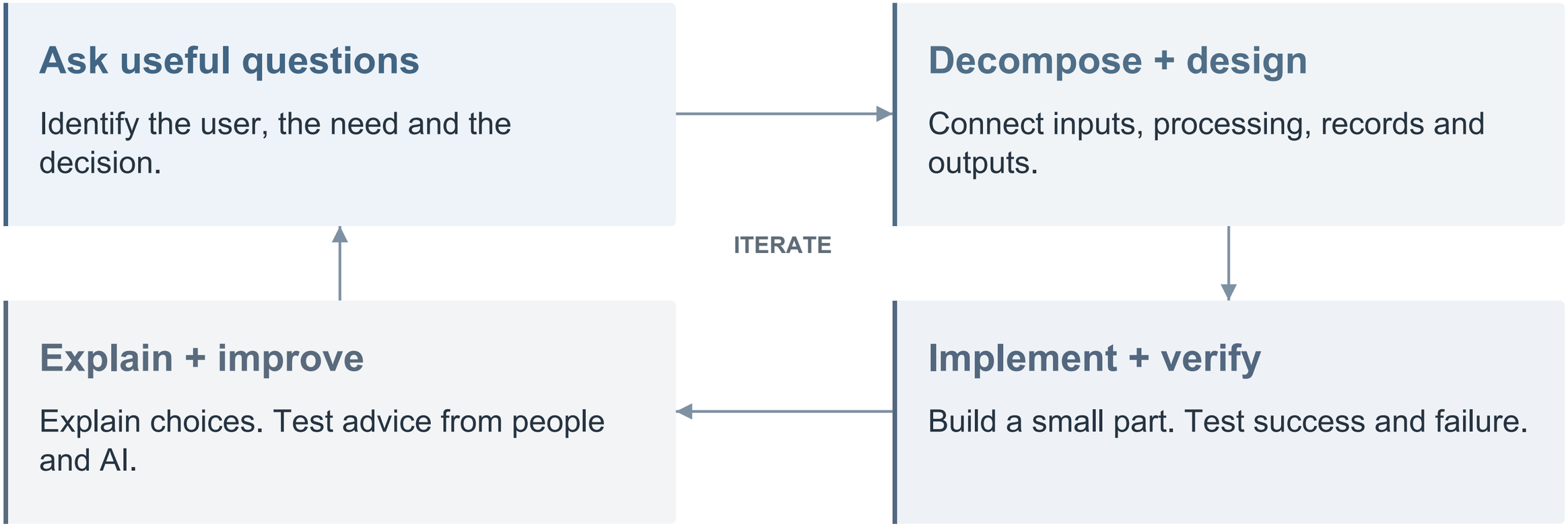
4

Check the result

Test the behavior and improve it with others.

Learn to turn people’s needs into systems they can trust.

Skills this course aims to develop



Ask clearly, design small parts, test your implementation and explain your judgment.

A system begins with people

ILLUSTRATIVE WORKPLACE CASE

A fictional Cambodian university team coordinates surplus-food pickups. We develop a Python + SQLite system to plan, reserve and track them.

Donor

What can we offer, and in what units?

Volunteer team

What has been reserved and collected?

Community partner

What was actually delivered?

The coordinator uses the system; people approve arrangements and carry out the pickups.

Our destination: a pickup coordination system

Coordinator

Uses the system to plan pickups and follow up missing records.

CAMPUS FOOD RESCUE · PYTHON + SQLITE

Offer list

Reviewed batches
Available containers

Pickup planner

Choose batch lines
Calculate + reserve

Pickup status

Record collection
Record delivery

SAVED RECORDS

Batch IDs · pickup IDs · line quantities
Historical units · collection / delivery events

REPORT + FOLLOW-UP

Stage totals
Pending pickups

Build and verify the operations in Python + SQLite; the dashboard is a concept view.

Course access and tools

The materials, help routes and working environment for this course

Know what to complete—and when to stop

Required

Complete the labelled weekly evidence and make sure you can explain it.

Already supplied

Use the starter, programs, data and diagrams. Do not rebuild the infrastructure.

Optional

Extra readings, examples and extensions can wait until required work is saved.

Save the required evidence and explain it; optional material can wait.

The course package is your starting point

1

README

Find the learning path, setup, current notebook, slides and ZIP.

2

Live class

Use the access details announced separately for the class.

3

Your notebook

Use Colab or a local Python environment.

4

Reflection

Follow the form link and select the week.

Start from README, then follow the current week’s notebook guidance.

Questions belong in the shared Issues space

1

Search

Look for the same week, topic or error.

2

Show

Give input, expected result and observed result.

3

Discuss

Add your OS, Python version and what you tried.

4

Close the loop

Try the suggestion and report what changed.

Ask during or after class. A clear question helps the next student too.

Choose one working environment

Google Colab

Use a browser and a Google account. Prepare the support files and save your own copy.

Your computer

Use Python 3 and JupyterLab. Keep the notebook, programs, diagrams and data together.

Prove it works

Run a cell, inspect the output, save and reopen. Report a small setup error early.

The same Python ideas apply on Windows, macOS, Linux and Colab.

Learning and participation

How class activities, reflection and attendance evidence fit together

Each week: learn Python, practise, then build

Session 1 · Python

Start with the programming topic.
Read syntax and trace short code
in the weekly notebook.

Session 2 · Practice

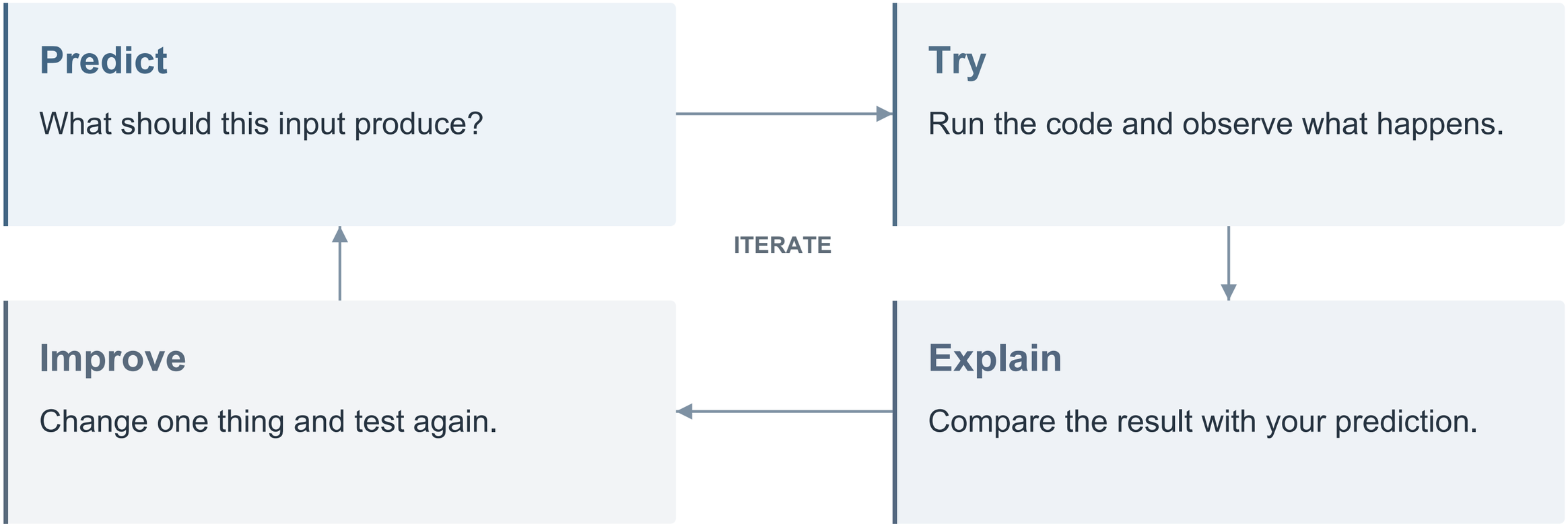
Use the same weekly notebook.
Predict, run and change examples.
Explain a result with a partner.

Session 3 · Project

Use a short design briefing.
Implement the weekly component.
Discuss, check and keep evidence.

Follow Start → Sessions 1–3 → Finish in one weekly notebook. Basics is optional lookup.

Learning together is an active cycle



A surprising result is a useful starting point for investigation.

Make a small-group discussion useful

Bring an example

Show a small input, the expected result and what actually happened.

Share the work

Rotate who runs the code, checks the result and reports the reasoning.

Leave a trace

Write one decision, one example and one question for the class.

Explain the reasoning so another person can continue the work.

If your connection drops

1

Keep your work

Save your notebook or script locally.

2

Use text

Use downloaded materials and keep a written question.

3

Reconnect

Rejoin using the announced class access when you can.

4

Follow up

Describe your participation honestly in the form.

A connection problem is something to explain, not a reason to invent progress.

A short reflection closes the week

1	**Identify the week**	Open the form from Course Links; choose Week 1–10.
2	**Describe your participation**	Report live, partial, independent or no participation honestly.
3	**Show one learning step**	Name an example, a result, a question or a next step.
4	**Help improve the course**	Give brief feedback on pace, clarity and support.

A small, specific reflection is more useful than “I understood everything.”

Attendance evidence and feedback

Participation record

Your name, student ID and selected week help match the response to the class roster.

Instructor review

The instructor checks records and follows up before confirming attendance status.

Feedback with identity

This form is identified, not anonymous. Use the private course route for sensitive concerns.

Submitting the form provides evidence; it does not alone prove attendance.

Before the first programming activity

Check	What success looks like
Course package	You can open README and the current week.
Live class	You have the current access details announced for the class.
Python or Colab	A code cell runs and shows its output.
Save + reopen	Your own file opens with your changes intact.
Help route	You can keep a written question and email submit.progmis@gmail.com.

Tell the instructor about a setup problem early, with the error message.

Assessment and projects

What counts, when it happens and what evidence students keep

Proposed assessment structure

Component	Weight	Main evidence
Participation and attendance	10%	Attendance record, participation and weekly reflection
Six research assignments	30%	Six individual submissions worth 5% each
Individual midterm	30%	Week 4 programming task, tests and explanation
Final project	30%	Scoped improvement, tests, handoff and individual evidence

The four proposed components total 100%; university confirmation still applies.

Six research assignments

Research / topic	Assigned	Shared
1 · Goals + systems	Week 1	Week 2
2 · Functions + tests	Week 2	Week 3
3 · Input validation	After Week 4 midterm	Week 5
4 · Persistent records	Week 5	Week 6
5 · Failure handling	Week 6	Week 7
6 · Output checks	Week 7	Start of Week 8

Proposed 5% each · no research submission/sharing in Week 4 · none in Weeks 9–10.

Research evidence and responsible AI use

Investigate

Use credible sources, identify the question and check claims before relying on them.

Discuss with AI, if used

Ask follow-up questions or challenge an answer. Keep a link, screenshots or relevant excerpts.

Explain your reasoning

State what you understand, how the evidence supports it and what remains uncertain.

Full marks do not depend on AI, a paid tool or a presentation.

Research sharing and marks

Normal full marks

Submit clear evidence and your own reasoning in slides, a short report or another readable format.

Optional volunteer sharing

An optional sharing bonus may be offered after its cap and fair opportunity rules are confirmed.

Fair opportunities

Any bonus must come with an announced equitable opportunity plan.

The written or visual submission can earn the normal full 5% without presenting.

Proposed individual midterm

Build and test

Complete one planning function and pass four common cases for totals, review precedence and the boundary.

Package

Submit README.md, midterm.py and test_midterm.py in a ZIP another person can run.

Explain

Run and explain your own code. State whether and where AI helped, or state that it was not used.

The proposed 30-point midterm uses Weeks 1–4 Python, a reproducible ZIP and individual explanation.

Your final project starts before the final week

1

Week 8 · Propose

Each person pitches a need. Your team selects a small change and two acceptance cases.

2

Week 9 · Build

Use the supplied baseline. Implement, test, exchange peer feedback and retest.

3

Week 10 · Present

Demonstrate, hand over the system and complete individual verification.

The proposed 30% final project rewards a small verified improvement and usable handoff.

Week 10: show a system another person can use

Explain the change

Who needs it? What is in scope? Show the baseline and your approved improvement.

Demonstrate evidence

Run acceptance cases and a boundary or failure case. Let a peer try an unseen input.

Hand it over

Provide starting data, run instructions, expected results, recovery advice and one limitation.

Present the need, demonstrate the evidence, and explain a limitation.

Find the task—and check the current announcement

Weekly brief

One weekly assignment PDF
Task, example and required evidence.
Research appears when assigned.

Assessment briefs

Separate Midterm and Final PDFs
Contracts, tests, deliverables
and assessment criteria.

Submission details

Submission: submit.progmis@gmail.com.
Deadlines and exact file names will be announced.
Keep your work and a backup.

Open assignments/README.md. Weekly briefs organize existing work; they add no new grading component.

Start the course

A first conversation and optional references for the next step

Our first system question

ILLUSTRATIVE WORKPLACE CASE

Imagine the volunteer desk receives “four boxes available.” With a partner or in the class chat, identify one missing detail and one useful first operation.

Notice

Who needs information from whom?

Question

Which detail is missing or ambiguous?

Start small

What could a first working program calculate or record?

Connect a person’s need to explicit information, a small operation and a check.

Further reading in English

1. ## Python Tutorial — introduction and first examples

   https://docs.python.org/3/tutorial/

2. ## Jupyter Notebook — running code

   https://jupyter-notebook.readthedocs.io/en/stable/examples/Notebook/Running%20Code.html

3. ## GitHub Docs — creating an issue

   https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue

Read one relevant section, then bring an example and a question.